

ICT1011: Computer Organisation and Architecture

Tutorial 3: Machine Code and Addressing Modes

Q1. Addressing Modes

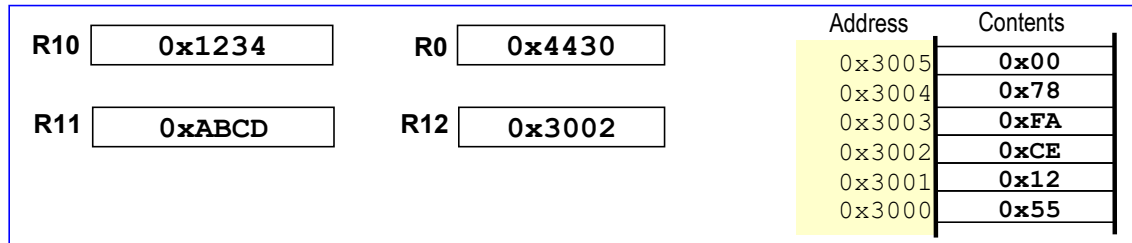


Figure Q1

With reference to Figure Q1, give the full hexadecimal contents on **any registers** (i.e. R0-R15) and memory locations that are altered by the execution of each of the instruction given below. Assume the initial CPU state prior to the execution of each instruction is shown in Figure Q1.

- (a) **MOV.B R10,R11** (b) **MOV.W R12,R11** (c) **MOV.W #-1,R11**
- (d) **MOV.W &3003H,R11** (e) **MOV.W @R12,R11** (f) **MOV.W @R12+,R11**
- (g) **MOV.B 2(R12),R11** (h) **MOV.W &3004,0(R12)**

Q2. Assembly Instructions and Machine Codes

Convert the following assembly language instructions to its machine code form:

- (a) **ADD.W R10, R9**
- (b) **MOV.W #0, R9**
- (c) **MOV.B &5566H, R9**
- (d) **MOV.W &2233H, &5566H**
- (e) **PUSH.W R1**
- (f) **RETI**

Q3. Address Modes Operations

(a) Determine the length and number of execution cycles needed to execute the instructions:

(i) **mov.w R10,R11**

(ii) **mov.w @R10,R11**

Despite the fact that the data size being moved is word-sized in both cases, why does instruction (ii) take one extra state to execute compared to instruction (i)?

(b) Comment on the two segments of code shown in Fig Q3a and Fig. Q3b given that the results in R10, R11 and R12 are similar after execution of each set of the instructions.

```
mov.w #0ABCDH,R10  
mov.w #0ABCDH,R11  
mov.w #0ABCDH,R12
```

Figure Q3a

```
mov.w #0ABCDH,R10  
mov.w R10,R11  
mov.w R10,R12
```

Figure Q3b

(c) Based on the instruction **mov.w #0, R11**, describe the optimisation provided by the Constant Generator of the MSP430. (Hint: look at the machine codes with and without a constant generator)